

Managing the Shard Tree

Agents, DevOps, and the automation crossover — a management-economics companion to occupancy-driven sharding in F1R3FLY

F1R3FLY Industries — working note

June 15, 2026

A short note, and a companion to “Rent and Shard Splitting.” That note priced running a shard tree; this one prices operating it—who provisions the nodes, watches the tables fill, fires the split, migrates the cold tenants, and answers the pager at 3 a.m. The arithmetic in §4 is an engineering estimate built on public 2026 figures (Anthropic API list prices, OCI GPU list prices, and US SRE/DevOps compensation surveys); every assumption is a lever in the companion workbook so the numbers can be re-derived or re-priced.

1 The problem in one paragraph

A single F1R3FLY shard is ten nodes costing $C \approx \$4,160/\text{month}$ (“Rent and Shard Splitting,” §5). But a shard is never alone for long: as occupancy ratchets toward the split threshold α^* , the tree fans out, and the sector roadmap in the sharding-economics workbook carries the AI engine alone from one shard to ~ 334 by year five, with the cross-shard *connector* layer reaching $\sim 12,000$ shards at maturity. Each of those shards must be stood up, monitored, split, and—per the rent lifecycle—have its tenants demoted and evicted on schedule. That operational work can be done by a human DevOps team, by an autonomous agent, or by a mix. The three differ not in *what* they do but in *how their cost scales*: human cost is a step function with a high fixed floor and a steep per-shard slope; agent cost is nearly flat, a fixed platform charge plus pennies of inference per shard. Where those curves cross is the whole question, and it lands at a surprisingly small tree.

2 Three regimes

We cost three ways of operating a tree of S homogeneous shards.

(1) Full automation. An agent provisions hardware, stands up nodes, and manages the whole tree—health, splits, rent decisions, tenant migration. No in-house operators; physical failures are absorbed by the cloud provider’s own hardware replacement, and a thin premium-support retainer covers SLA disputes. This is the cloud-native limit. It is a *price floor*, not a deployable plan, because an agent cannot currently re-seat a failed DIMM or litigate a credit with a carrier; we cost it anyway, because it is the asymptote the other two are measured against.

(2) Agent in the mix (hybrid). The agent does the routine—all monitoring, provisioning, split and migration decisions—and escalates the exceptions to a thin human team sized by the *exception rate*, not by the shard count. This is the realistic deployment.

(3) DevOps team. Humans provision and manage the tree with conventional infrastructure-as-code. The agent platform is not built; cost is people.

3 The cost models

Let W be the fully-loaded monthly cost of one SRE, σ a span of control (shards one engineer can carry), φ a coverage floor (minimum headcount for a 24/7 rotation), P the fixed cost of *having* an ops agent (the engineers who build and maintain the harness, evals, and guardrails, plus telemetry), ρ a premium-support retainer, and a the agent’s inference cost to manage one shard for one month. Then

$$\text{DevOps: } M_d(S) = W \cdot \max(\varphi_d, \lceil S/\sigma_d \rceil), \quad (1)$$

$$\text{Full-auto: } M_f(S) = P + \rho + a S, \quad (2)$$

$$\text{Hybrid: } M_h(S) = P + a S + W \cdot \max(\varphi_h, \lceil S/\sigma_h \rceil). \quad (3)$$

The structure is the entire story. Equation (1) is flat at $W\varphi_d$ until the tree outgrows the on-call floor, then climbs one engineer per σ_d shards—marginal cost W/σ_d per shard. Equation (2) is a fixed intercept plus a gentle slope a . Equation (3) pays *both* a platform intercept and a human term, but the human term uses $\sigma_h \gg \sigma_d$ because the agent has absorbed everything routine: one operator now backstops hundreds of shards instead of dozens.

3.1 What the agent’s a actually is

The per-shard inference cost is a transparent token sum over the agent’s activities k (routine monitoring, hard decisions, provisioning), each with n_k calls per shard-month, input/output token counts $\tau_k^{\text{in}}, \tau_k^{\text{out}}$, a cached-input fraction β_k , and model prices $p_k^{\text{in}}, p_k^{\text{out}}$ per million tokens with cache multiplier γ (prompt caching bills cached input at 10%):

$$a = \sum_k n_k \frac{[\beta_k \gamma + (1 - \beta_k)] \tau_k^{\text{in}} p_k^{\text{in}} + \tau_k^{\text{out}} p_k^{\text{out}}}{10^6}.$$

With routine monitoring on Haiku 4.5, hard decisions (split / migrate / triage) on Opus 4.8, and provisioning on Sonnet 4.6 at June 2026 list prices [2], the workbook returns

Table 1. Agent inference cost to manage one shard for one month.

Activity	Calls/shard-mo	Model	Cost (\$/shard-mo)
Routine monitoring	720	Haiku 4.5	\$2.61
Hard decisions	6	Opus 4.8	\$1.40
Provisioning / stand-up	0.5	Sonnet 4.6	\$0.17
Total a			\$4.17

Four dollars and change. Against an infrastructure bill of $C \approx \$4,160$ per shard, the agent’s management overhead is *one tenth of one percent*. The same shard managed at the margin by a human costs $W/\sigma_d \approx \$667/\text{month}$ —about 16% of infra, and roughly 160 times the agent. That ratio, not any subtlety of the curves, is why the crossover exists and why it is early.

4 The crossover

Parameters. Loaded SRE $W = \$20,000/\text{month}$ (a \$150,000 total-comp engineer at a 1.6× loading factor [3, 4]); DevOps span $\sigma_d = 30$ shards/SRE with a lean 24/7 floor $\varphi_d = 2$; hybrid span

$\sigma_h = 300$ with exception floor $\varphi_h = 1$; agent platform $P = \$28,000/\text{month}$ (1.25 platform FTEs plus \$3,000 tooling); retainer $\rho = \$2,000$. All are levers; §6 sweeps the ones that move the answer.

Table 2. Monthly cost to manage the whole tree. “Feasible best” chooses between the two deployable regimes (DevOps, Hybrid); Full-auto is shown as the floor.

Shards S	DevOps	Full-auto (floor)	Hybrid	Feasible best
1	\$40,000	\$30,004	\$48,004	DevOps
25	\$40,000	\$30,104	\$48,104	DevOps
50	\$40,000	\$30,208	\$48,208	DevOps
60	\$40,000	\$30,250	\$48,250	DevOps
90	\$60,000	\$30,375	\$48,375	Hybrid
100	\$80,000	\$30,417	\$48,417	Hybrid
250	\$180,000	\$31,042	\$49,042	Hybrid
334	\$240,000	\$31,392	\$69,392	Hybrid
1,000	\$680,000	\$34,166	\$112,166	Hybrid
5,000	\$3,340,000	\$50,832	\$388,832	Hybrid
12,074	\$8,060,000	\$80,305	\$898,305	Hybrid

Where it crosses. Below ~ 60 shards a lean human team is the cheapest *deployable* option: the tree is small enough that two on-call engineers cover it, and that \$40,000 undercuts the \$48,000 floor of standing up *and* owning an agent platform. The flip comes the moment DevOps must hire its third engineer—at $S > 2\sigma_d = 60$ —after which human cost climbs a step every thirty shards while hybrid barely moves. Dropping the ceilings and floors, the crossover has a clean closed form:

$$S^* \approx \frac{P + W}{W/\sigma_d - a} = \frac{28,000 + 20,000}{20,000/30 - 4.17} \approx 72 \text{ shards.}$$

The denominator is the per-shard *advantage* of an agent over a human (\$667 − \$4); the numerator is the fixed price of admission to automation. Both sides are small numbers, so the quotient is small: F1R3FLY crosses into agent-favourable territory while its tree is still in the low dozens—well inside the year-four AI ramp, and an order of magnitude before the connector layer exists at all.

The two asymptotes. At the high end the regimes separate brutally. At the connector-scale 12,074-shard tree, a pure DevOps organisation needs 403 engineers and \$8.06M/month; the hybrid model needs 41 (one platform team plus exception coverage) and \$0.90M—11 % of the DevOps bill; full automation, were it feasible alone, would cost \$80,000, or 1 %. As a fraction of the \$50M infrastructure spend at that scale, human-bound management is a persistent 16 % tax, while agent-led management falls below 2 % and keeps falling. Management is a major line item exactly when it is human and the tree is large; it is a rounding error exactly when it is automated.

Remark 1 (The agent is already in the design). *The hard decisions priced on Opus above are not generic “ops.” They are the very events the rent companion specifies: when to split (the α^* test), which tenants to demote and how far (the bid-rent sort), and when to evict (the depletion cascade). These are local, state-dependent, content-addressed decisions over a tuplespace—precisely the shape a reactive agent handles well, and precisely the shape that scales badly across a human*

on-call rotation. The split threshold and the management model are the same artefact seen twice: occupancy fires the split; the agent executes it; rent prices the migration. Automating the tree is not a new system bolted on, but the reducer’s own lifecycle given an operator.

5 Why hybrid, and what stays human

Full automation wins on dollars at every scale (Table 2, floor column) for the same reason it is not deployable: it books no humans. The residual that keeps humans in the loop is small but irreducible, and it is physical or adversarial rather than computational:

- **Hardware the provider won’t touch.** On managed cloud the carrier swaps failed disks and hosts; on owned or co-located metal someone must. This is the difference between ρ (a support line item) and an on-site hands contract.
- **SLA and contract disputes.** Negotiating a credit for a region outage, or a commitment discount, is a human-to-human commercial act. The agent can assemble the evidence; a person signs.
- **Novel incidents and change authority.** The first occurrence of a failure mode, and any irreversible action (key rotation, consensus reconfiguration), wants a human in the approval path until the agent’s eval coverage earns autonomy.

Hybrid prices exactly this: it pays the agent’s flat aS for everything routine and keeps a team sized by how often the above fires—captured here as the tenfold span $\sigma_h = 300$. Raise the exception rate (owned hardware, regulated change control, immature evals) and σ_h falls toward σ_d , pulling hybrid back toward DevOps; lower it (fully managed cloud, mature autonomy) and σ_h rises, pulling hybrid toward the full-auto floor. The single parameter σ_h is the dial between “agent with human backup” and “humans with agent help.”

6 Sensitivity: the levers that move the answer

The DevOps floor. The crossover sits at ~ 60 shards *because* we modelled a lean two-person rotation. A team holding a strict 24/7 SLA at $\varphi_d = 4$ pays \$80,000 from the first shard, and hybrid wins essentially everywhere; conversely a single-engineer hobby deployment ($\varphi_d = 1$) pushes the crossover out past 100 shards. The floor is the most consequential single assumption, and it is a policy choice (what coverage you promise), not a technical one.

The agent platform P . Halving the platform cost (a shared agent across many trees, amortised tooling) moves S^* down proportionally and makes hybrid competitive against even a one-person team. Doubling it (bespoke harness, heavy eval burden) roughly doubles the crossover. P is a one-time-ish capability cost; the more trees it serves, the lower its effective per-tree share.

Inference price and model mix. The agent slope a is dominated by routine monitoring on Haiku; pushing more decisions to Opus, or polling more often, raises it, but it would have to rise by two orders of magnitude—to $\sim \$667/\text{shard}$ —before it matched the human marginal cost. There is enormous headroom: prompt caching, batch settlement of non-urgent checks, and cheaper routine models all cut a further. The agent’s cost advantage is not fragile.

Span of control. Better human tooling raises σ_d and delays the crossover; it does not remove it, because the human curve still has positive slope and the agent curve is essentially flat. No achievable σ_d closes a gap that is two orders of magnitude per shard at the margin.

7 Self-hosting the agent’s model: local inference

Everything so far priced the agent’s inference at third-party API rates—pure variable cost, a few dollars per shard, zero fixed outlay. Dedicating hardware to a *local* model inverts that structure. A GPU runs whether it serves one shard or three thousand, so local inference is a *fixed step cost*: the same shape as the human team, one level down the stack. The question is the same too—at what tree size does the fixed asset amortise below the variable bill?

The commodity tier. Split the agent’s work by volume. Routine monitoring is high-volume (~ 720 calls/shard-month) and cognitively cheap—exactly what a small open-weight model on a commodity inference GPU does well. On an OCI inference shape (an L40S- or A10-class card at $\sim \$1.50$ /GPU-hour list, $\sim \$1,095$ /month; $\sim \$0.85$ on a three-year commit [5]) running a quantised model at a conservative 5,000 tokens/s and 70% utilisation, one GPU clears ~ 9.2 billion tokens/month. The routine workload is 3.24 M tokens/shard-month, so a single card saturates at $\sim 2,840$ shards.

Where local beats the API. Per-shard local cost is the GPU bill divided by the tree it serves; it falls as the card fills. It crosses the flat API price ($\$2.61$ /shard for routine on Haiku 4.5) at the *break-even tree size*

$$S_{\text{BE}} = \frac{\text{GPU \$ / month}}{\text{API \$ / shard}} = \frac{1,095}{2.61} \approx 420 \text{ shards.}$$

Below 420 shards the card sits mostly idle and the API is cheaper; above it, local routine cost collapses toward $\sim \$0.44$ /shard once the GPU saturates—about 17% of the API price, a sixfold cut on the routine tier.

Table 3. Routine-tier inference: per-shard cost, API vs self-hosted GPU.

Shards S	GPUs	Local \$/shard	API \$/shard	Cheaper
100	1	\$10.95	\$2.61	API
334	1	\$3.28	\$2.61	API
420	1	\$2.61	\$2.61	break-even
500	1	\$2.19	\$2.61	Local
1,000	1	\$1.09	\$2.61	Local
2,500	1	\$0.44	\$2.61	Local
12,074	5	\$0.45	\$2.61	Local

Why the frontier tier stays rented. The hard decisions (split, migrate, triage) run on a frontier model precisely because quality there is load-bearing, and they are *low* volume (~ 6 /shard-month). Self-hosting frontier quality means a large open-weight model on a multi-GPU node—an $8 \times \text{H100}$ box lists near $\$58,400$ /month [5]—against a hard-decision API bill of only $\sim \$16,800$ /month even at the 12,074-shard connector scale. The dedicated floor is over three times the entire bill it would

replace, and open-weight quality may still trail Opus. The economical architecture is therefore two-tier and mirrors the human story exactly: *own the commodity high-volume tier, rent the exceptional low-volume one.*

What it changes, and what it doesn't. Self-hosting does *not* move the agent-versus-DevOps crossover of §4. That crossover sits near 60 shards, far below the 420-shard inference break-even, so in the contested region one is on the API regardless; the management decision is unaffected. What local inference does is *deepen* the agent's lead once the tree is large: the blended per-shard agent cost falls from \$4.17 to $\sim$ \$2.01 at connector scale, and the full-auto price floor drops from \$80,305 to \$54,310/month—a 32% cut—because the single largest variable component, routine inference, has been converted to a nearly-flat asset.

Remark 2 (Below break-even, non-cost drivers can still decide it). *Cost is not the only axis. Keeping the management model local holds operational telemetry—shard health, tenant patterns, traffic—off a third-party endpoint, which is a data-residency and sovereignty argument for the regulated engagements (the VA cohort, Tata, fintech), where the compliance multipliers in the sector model already show these tenants paying for assurance. It also shortens the control-loop round trip. Any of these can justify a dedicated GPU below the 420-shard break-even, where it loses on dollars alone. And there is a F1R3FLY-native option the generic analysis misses: the model can run as a tenant on the tree it manages—co-resident, metered in phlogiston, paying rent like any other residual—so the operator becomes self-hosted inside the very substrate it operates, rather than a service bolted on beside it.*

8 Do you even need the model? Algorithmic control and the three-tier hierarchy

Everything above assumed that automating the tree means an LLM does the deciding. That assumption is wrong, and worth correcting. Most shard-tree decisions are *deterministic*: split when $\alpha > \alpha^*$, demote a tenant when its rent fund empties, provision a shard when capacity is hit, remediate when a health metric leaves its band. These are threshold tests and B-tree range handoffs—exactly the R1–R5 rewrite families of the rent companion—and a conventional control plane (operators, reconcilers, policy loops) executes them with no model in the loop. The earlier regimes priced *all* $\sim$ 726 decisions per shard-month through Haiku or Opus; that treats the LLM as the control plane rather than as the exception handler above it, and over-prices inference by roughly twentyfold.

Tier 0: the algorithmic control plane. A deterministic controller costs a fixed engineering charge to build and maintain ($\sim$ \$25,000/month here) and essentially *nothing* per shard—the reconcile loop is CPU cycles on nodes you already run. Its fully automated floor is therefore *flat*: $\sim$ \$27,000/month for a tree of *any* size, from ten shards to twelve thousand. That is the true automation floor, and it sits below every LLM regime in this note, because it buys no tokens at all.

Tier 1: the model as escalation, priced on the residual. The LLM's right job is the fraction of decisions the rules do not settle—genuinely ambiguous triage, novel failure modes, multi-factor policy calls. If a share $\delta \approx 0.95$ of decisions is deterministic, the model sees only the residual, and its inference cost falls from \$4.17 to $a_{\text{resid}} \approx$ \$0.20 per shard-month. The LLM is to the rules what the human is to the LLM: each tier is the exception handler of the tier below it.

Tier 2: the human. Unchanged—the physical and commercial residual of §5.

Table 4. Four ways to run the tree. Floors carry no humans; “feasible best” chooses between the two deployable regimes.

S	DevOps (no auto)	Algo full-auto (floor)	Algo + human (no LLM)	Three-tier (algo+LLM+human)	Feasible best
50	\$40,000	\$27,000	\$45,000	\$53,010	Algo + human
150	\$100,000	\$27,000	\$45,000	\$53,030	Algo + human
250	\$180,000	\$27,000	\$65,000	\$53,050	Three-tier
1,000	\$680,000	\$27,000	\$165,000	\$113,200	Three-tier
5,000	\$3,340,000	\$27,000	\$705,000	\$374,000	Three-tier
12,074	\$8,060,000	\$27,000	\$1,645,000	\$855,416	Three-tier

What the regimes say. Among floors, algorithmic full-auto (\$27,000 flat) undercuts the LLM full-auto floor at every scale—no surprise, since it runs no inference. Among *deployable* options, the ranking flips with scale. Below ~ 150 shards a rules-only plane with a single on-call engineer is cheapest: when one human already covers the escalations, the LLM tier’s platform ($\sim \$8,000/\text{month}$) and inference are not worth their keep. Above ~ 250 shards the three-tier system wins, and the gap widens fast: the model roughly doubles human reach (σ from 150 to 300), and at scale that saves *multiple* \$20,000/month engineers for a few thousand dollars of platform and pennies of residual inference. The crossover is the point where a no-LLM team must hire its second escalation engineer. Notably the correctly-scoped three-tier even beats the all-LLM “hybrid” of §4 (\$855k vs \$898k at connector scale), because deterministic decisions no longer pay for tokens.

The honest conclusion. The cheapest automation is mostly *rules*, with a thin model layer on top and a thinner human layer above that. The LLM earns its place precisely in the band of decisions that are too ambiguous for a deterministic rule yet not physical—and that band, not the whole workload, is what it should be priced on. None of this moves the automation-versus-DevOps crossover near 60 shards; it lowers the automation floor and corrects the architecture. There is a F1R3FLY reading too: the control plane is the reducer’s own lifecycle—the splits, demotions, and migrations are rewrites the rho machine already performs—so Tier 0 is less a new system than the normalisation dynamics given an actuator, with the model invoked only where the rewrite rules leave genuine policy slack.

9 One open operational knob

The model treats provisioning as an amortised monthly episode. In a tree that is actively splitting, provisioning is bursty—a wave of stand-ups follows every density event—and the agent’s value is highest precisely there, where a human team would be paged into a scramble. The incentive-compatible refinement is to let the split-overhead K from the rent companion *also* price the management burst: a split that is cheap to coordinate (large K , well-separated join graph) is cheap to *automate*, while a split that fragments a hot join graph is both consensus-expensive and attention-expensive. We flag this coupling—management cost as a function of K , not a flat per-shard charge—as the natural next iteration, and leave it as a decision rather than settling it here.

Pieces assembled: three operating regimes and their cost laws (§2, §3); a transparent token build-up of the agent’s per-shard cost (Table 1); the crossover at ~ 60 shards with its closed form and

its two asymptotes (§4); the case for hybrid and the irreducible human residual (§5); a sensitivity sweep over the levers that actually move it (§6); and the nested local-vs-API inference crossover when hardware is dedicated to a self-hosted model (§7); and the pure-algorithmic control plane with the model recast as a residual escalation tier in a three-tier hierarchy (§8). Live model: the Mgmt Assumptions, Agent Cost, Mgmt Crossover, Local Inference, and Algorithmic vs LLM tabs of f1r3fly_sharding_economics.xlsx.

References

- [1] F1R3FLY Industries. Rent and shard splitting: a storage-economics companion to occupancy-driven sharding in F1R3FLY. Working note, 2026.
- [2] Anthropic. Claude API pricing (Haiku 4.5, Sonnet 4.6, Opus 4.8); prompt-caching and batch discounts. <https://docs.claude.com/en/docs/about-claude/pricing>, accessed June 2026.
- [3] Built In. Site Reliability Engineer and DevOps Engineer salary in US, 2026 (avg. total comp \$147,000–\$150,000). <https://builtin.com/salaries>, accessed June 2026.
- [4] KORE1. Site Reliability Engineer salary guide 2026 (base \$130,000–\$210,000; staff/principal \$300,000+ all-in). <https://www.kore1.com/sre-salary-guide-2026/>, accessed June 2026.
- [5] Oracle Corporation. Oracle Cloud Infrastructure pricing (compute, block volume, networking). <https://www.oracle.com/cloud/price-list/>, accessed June 2026.